End-to-End Machine Learning on Retail Transaction Data

A Theory-and-Results Report on the Sample Superstore Dataset

Debarshi Chakraborty

September 1, 2026

Companion codebase: `EY Training/ml_superstore/`
Data: `EY Training/data/Sample - Superstore.csv` (9,994 order lines, 2014–2017, USA)

Abstract

This report documents an end-to-end machine learning study built on the Sample Superstore retail dataset. It follows the same five stages as the accompanying Jupyter notebooks and source package (`EY Training/ml_superstore/`): data ingestion and quality assessment, exploratory data analysis, feature engineering, time-series forecasting of monthly sales, and supervised learning for line-level profit prediction and loss detection. Unlike the notebooks, whose purpose is interactive exploration, this report is written to stand alone: every technique used is first introduced from first principles — the statistical or algorithmic theory it rests on — and only then applied to the data, with the resulting figures, tables and their business interpretation presented immediately afterwards. The headline findings are that (i) order discounting beyond roughly 30% is the dominant driver of loss-making order lines, far outweighing category, region or shipping mode; (ii) an additive Holt–Winters exponential smoothing model forecasts monthly sales with an out-of-sample MAPE of approximately 23%, competitive with a seasonal ARIMA model and clearly better than a seasonal-naive baseline; and (iii) a random-forest classifier trained on leakage-safe features identifies loss-making order lines with a PR-AUC of 0.96 on a strictly future (2017) hold-out year, giving a concrete, auditable basis for a discount governance policy.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Objective

Retail order-line data — one row per product sold within an order — sits at the intersection of three classical machine-learning problem types: descriptive analytics (what happened), predictive analytics (what will happen), and diagnostic/prescriptive analytics (why it happened, and what to do about it). This report uses the public *Sample Superstore* dataset to walk through all three, mirroring a realistic analytics engagement:

1. **Data quality and exploratory analysis** — establish that the data is trustworthy and characterise its statistical structure.
2. **Feature engineering** — turn raw transactional fields into model-ready predictors, with explicit attention to information leakage.
3. **Forecasting** — model and project the aggregate monthly sales time series.
4. **Supervised learning** — model two line-level targets, *profit* (regression) and *is a line loss-making* (classification), and interpret what drives them.

Each stage in this report is structured identically: a **Theory** section derives or states the relevant statistical/ML machinery from first principles, followed by a **Method** section describing how it was applied to this dataset, then **Results** with the produced figures and tables, and finally an **Interpretation** section connecting the numbers back to a business reading. This mirrors, chapter for chapter, the five numbered notebooks in `EY_Training/ml_superstore/notebooks/` and the reusable modules in `src/`.

1.2 Dataset

The dataset contains $n = 9{,}994$ order lines spanning 4 years (2014–01–03 to 2017–12–30), 793 unique customers and 1,862 unique products, organised into 5,009 distinct orders across the United States. Table 1.1 summarises the raw schema.

Table 1.1: Raw schema of the Sample Superstore CSV.

Field	Type	Description
Row ID, Order ID	id	Line and order identifiers
Order Date, Ship Date	date	Order placement and shipment dates
Ship Mode	categorical	Standard / Second / First Class, Same
Customer ID, Customer Name	id/text	Buyer identity
Segment	categorical	Consumer / Corporate / Home Office
Country, City, State, Postal Code, Region	geography	U.S. location fields
Product ID, Category, Sub-Category, Product Name	catalogue	3-level product hierarchy
Sales	numeric ($)	Line revenue
Quantity	numeric	Units sold
Discount	numeric $[0, 1)$	Fractional discount applied
Profit	numeric ($)	Line profit (can be negative)

1.3 Software and reproducibility

All analysis is implemented in Python 3.10 using `pandas`, `numpy`, `scikit-learn`, `statsmodels`, `matplotlib`/`seaborn`. Code lives in `EY_Training/ml_superstore/src/` as importable modules (`data_loader`, `features`, `forecasting`, `modeling`, `pipeline`), exercised interactively by six Jupyter notebooks and, for one-shot reproduction, by `python -m src.pipeline`. Every figure reproduced in this report was generated by that codebase and saved under `outputs/figures/`; every number quoted in the text is read directly from the corresponding CSV in `outputs/tables/` or the machine-written `outputs/REPORT.md`. No figure or statistic in this document was computed by hand.

1.4 Notation

Throughout, y_i denotes a target value for observation i, $\hat{y}_i$ its model prediction, $\mathbf{x}_i \in \mathbb{R}^p$ its feature vector, and n the sample size of the set under discussion (which may be a train, test, or full-data set, made explicit in context). For the monthly sales series we write y_t, $t = 1, \ldots, T$, for the value in month t. $\mathbb{E}[\cdot]$, $\mathrm{Var}(\cdot)$ and $\mathrm{Cov}(\cdot, \cdot)$ denote expectation, variance and covariance under whatever probabilistic model is being assumed at that point (population model for theory, empirical/plug-in estimator when applied to data).

Chapter 2

Data Quality and Exploratory Analysis

2.1 Theory

2.1.1 Data quality auditing

Before any modelling, three questions must be answered about a tabular dataset $D = \{(\mathbf{x}_i)\}_{i=1}^{n}$: (i) is every field typed and parsed correctly, (ii) is any value missing, and (iii) is any row duplicated. Formally, for a column X we report the *missingness rate* $\hat{\pi}_X = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}[x_i \text{ is NA}]$ and *cardinality* $|\{x_i\}|$. A duplicate row under a primary key k is any pair $i \neq j$ with $k_i = k_j$; if such pairs exist, the key does not uniquely identify a record and downstream joins or grouped aggregates will silently double-count. These checks are cheap and guard against exactly the class of error (a re-exported CSV with duplicated header rows, an encoding mismatch producing NA in a text field) that otherwise corrupts every later stage silently.

2.1.2 Descriptive statistics and distribution shape

For a numeric column X with observed values $x_1, \dots, x_n$, the sample mean $\bar{x} = \frac{1}{n}\sum_i x_i$ and variance $s^2 = \frac{1}{n-1}\sum_i (x_i - \bar{x})^2$ summarise location and spread, but retail money variables are rarely well described by them alone because they are *right-skewed*: many small transactions and a long tail of large ones. Skewness is measured by the third standardised moment,

$$\gamma_1 = \frac{\frac{1}{n}\sum_i (x_i - \bar{x})^3}{\left(\frac{1}{n}\sum_i (x_i - \bar{x})^2\right)^{3/2}}, \tag{2.1}$$

with $\gamma_1 > 0$ indicating a right tail. A common remedy that stabilises both skew and the variance-vs-mean relationship (heteroscedasticity) seen in multiplicative processes like sales is the $\log(1 + x)$ transform, used throughout this project on `Sales` (and, with a sign-preserving variant, on `Profit`) purely for *visualisation*; models are always trained on the original scale unless stated otherwise, since tree ensembles are invariant to monotone transforms and linear models' coefficients are more directly interpretable untransformed here.

2.1.3 Correlation

For two numeric variables X, Y, the Pearson correlation coefficient

$$\rho_{XY} = \frac{\operatorname{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}} \tag{2.2}$$

measures *linear* association, $\rho_{XY} \in [-1, 1]$. It is insensitive to non-linear relationships (e.g. a U-shape can have $\rho \approx 0$ despite strong dependence), which is precisely why Section 2.3.4 supplements the correlation matrix with a binned conditional-mean plot: discount and profit turn out to have a threshold-like, not linear, relationship.

2.1.4 Seasonal-trend decomposition (STL)

A time series y_t is classically modelled as the additive sum of a slowly-varying trend T_t, a periodic seasonal component S_t (period L, here $L = 12$ months), and a remainder R_t:

$$y_t = T_t + S_t + R_t. \tag{2.3}$$

STL (Seasonal-Trend decomposition using Loess, Cleveland et al., 1990) estimates T_t and S_t by alternating two local regression (LOESS) smooths: it removes a running estimate of T_t from y_t to expose the seasonal cycle, smooths within each seasonal sub-series (all Januaries, all Februaries, ...) to obtain a smoothly time-varying S_t, then smooths $y_t - S_t$ to update T_t; this is iterated to convergence. Unlike a single global seasonal index, STL lets the seasonal shape drift slowly over the four years of data, and unlike classical decomposition it is robust to outliers when the `robust` option down-weights large residuals between iterations. We use STL purely as a diagnostic here; the forecasting models of Chapter 4 are fitted directly on y_t, not on the decomposed components.

2.2 Method

The raw CSV is read with `src.data_loader.load_raw` (encoding auto-detection across UTF-8/Latin-1/CP1252) and typed with `clean`, which parses `Order Date`/`Ship Date`, strips whitespace from every text column, drops exact and `Row ID` duplicates, and derives `Ship Days` = Ship Date − Order Date. The quality audit, univariate/bivariate plots and STL decomposition are produced by notebooks `01_data_overview.ipynb` and `02_eda.ipynb`.

2.3 Results

2.3.1 Data quality

The audit (`outputs/tables/01_data_quality.csv`) finds **zero missing values in any of the 21 raw columns**, no fully duplicated rows, and no duplicate `Row ID`s after cleaning — the data is analysis-ready without imputation. `Order Date` spans 2014-01-03 to 2017-12-30 (1,237 distinct dates); `Ship Days` is always non-negative, i.e. no order ships before it is placed.

2.3.2 Distributions

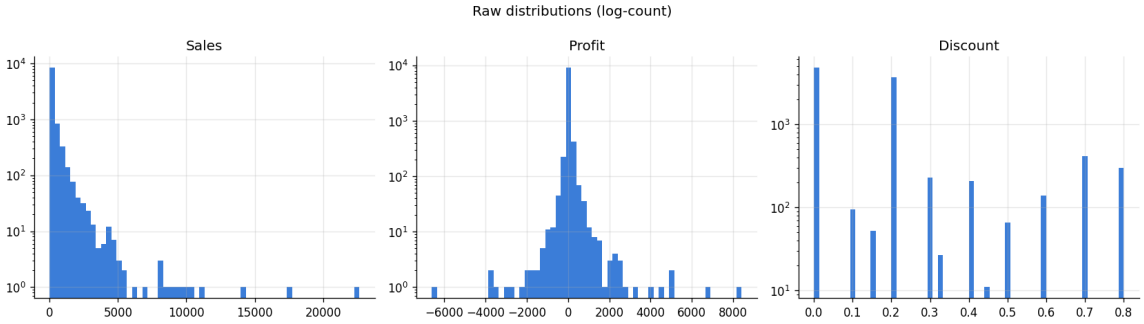

Figure 2.1: Histograms (log-count y-axis) of `Sales`, `Profit` and `Discount` on the raw scale, showing pronounced right skew in `Sales` and a bimodal, discount-tier structure in `Discount`.

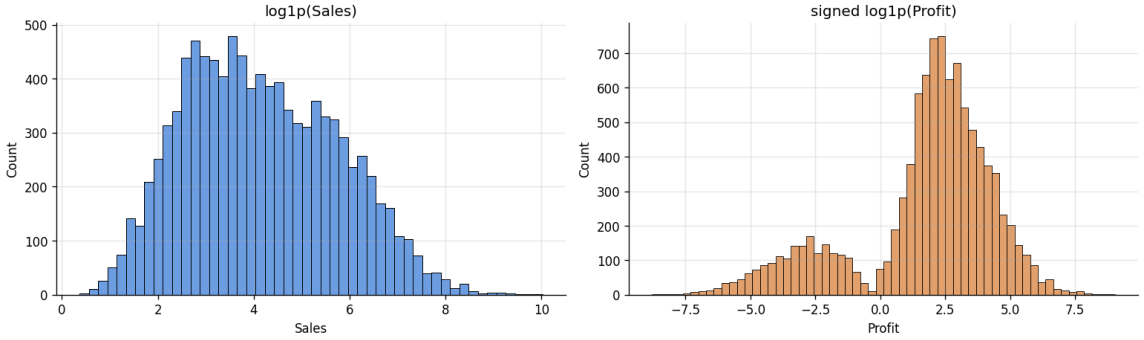

Figure 2.2: Left: $\log(1+\text{Sales})$ recovers an approximately unimodal, near-symmetric shape (Eq. (2.1) in the raw scale is large and positive; after the transform it is close to zero). Right: signed $\log(1+|\text{Profit}|)$, showing the loss-making mass as a distinct negative lobe rather than a long left tail.

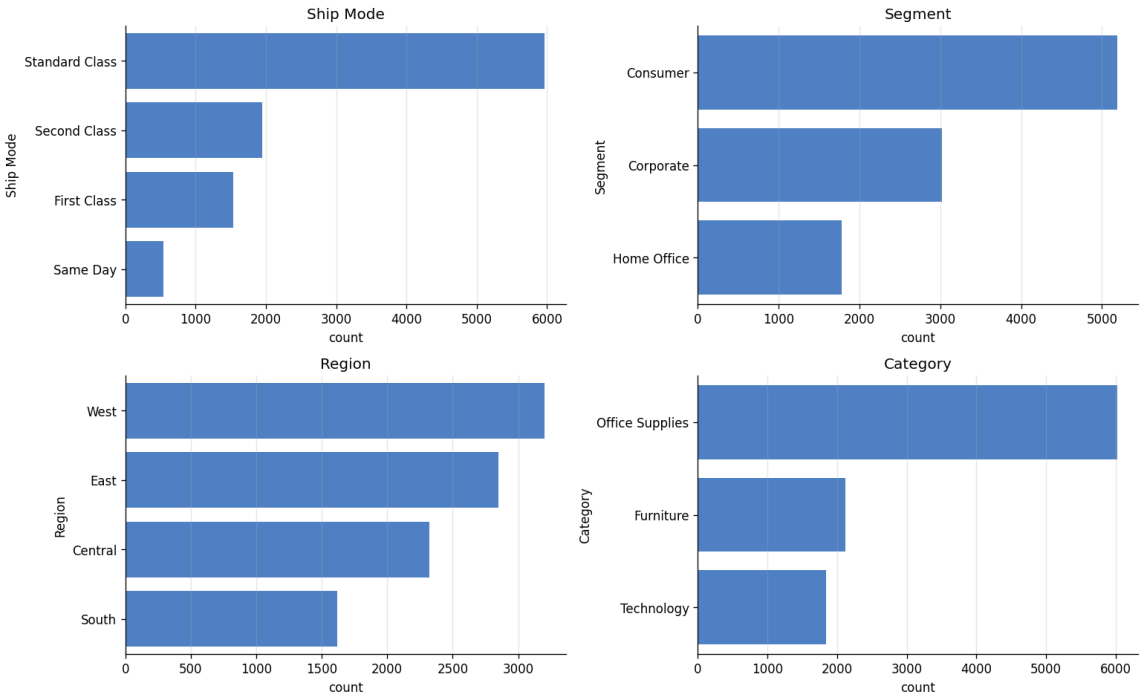

Figure 2.3: Order-line counts by `Ship Mode`, `Segment`, `Region` and `Category`. Standard Class and the Consumer segment dominate; the four regions are comparably sized, with West largest.

2.3.3 Profit by product hierarchy

Table 2.1 (full detail in `outputs/tables/02_by_subcategory.csv`) aggregates profit by sub-category. **Tables** lose $17,725 overall despite $207k of sales, and **Bookcases** lose $3,473 — both at average discounts above 21%, well above the store-wide mean. **Supplies** is also marginally loss-making. Every other sub-category is net profitable, led by Copiers (37% margin) and Phones ($330k sales, 13% margin).

Table 2.1: Selected sub-categories by total profit (ascending). Full table has all 17 sub-categories.

Category	Sub-Category	Sales ($)	Profit ($)	Avg. Discount	Margin
Furniture	Tables	206,966	−17,725	26.1%	−8.6%
Furniture	Bookcases	114,880	−3,473	21.1%	−3.0%
Office Supplies	Supplies	46,674	−1,189	7.7%	−2.5%
	… (14 profitable sub-categories) …				
Technology	Phones	330,007	44,516	15.5%	13.5%
Technology	Copiers	149,528	55,618	16.2%	**37.2%**

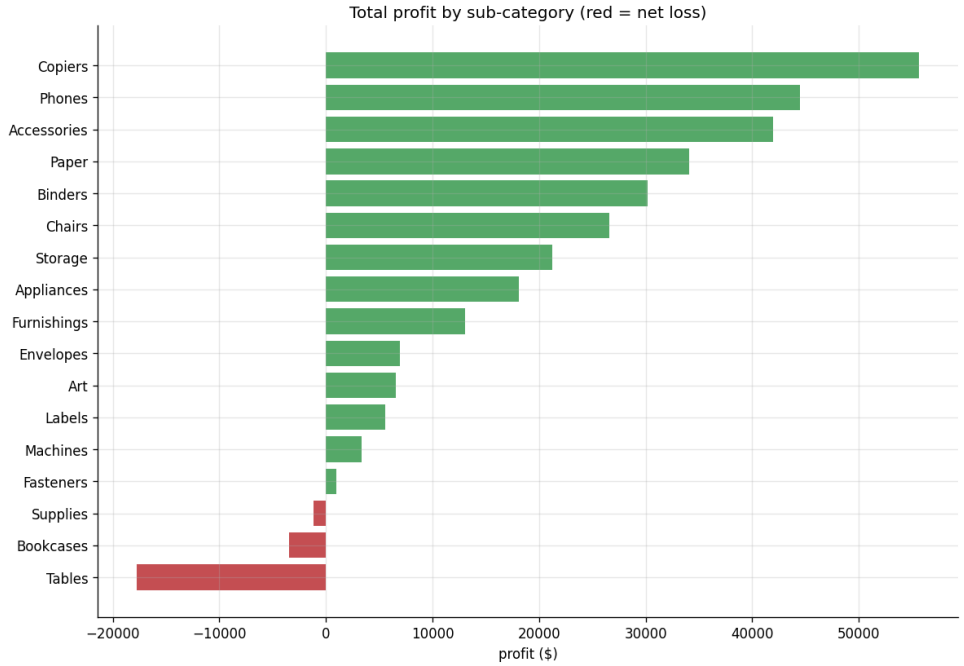

Figure 2.4: Total profit by sub-category; red bars are net loss-makers.

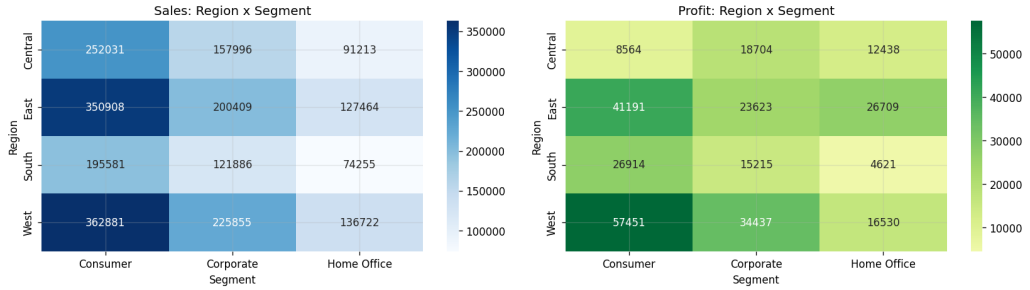

Figure 2.5: Sales (left) and profit (right) cross-tabulated by Region and Segment. Sales are fairly even across cells; profit is not — the Central/Consumer cell is comparatively weak.

2.3.4 Discount versus profit

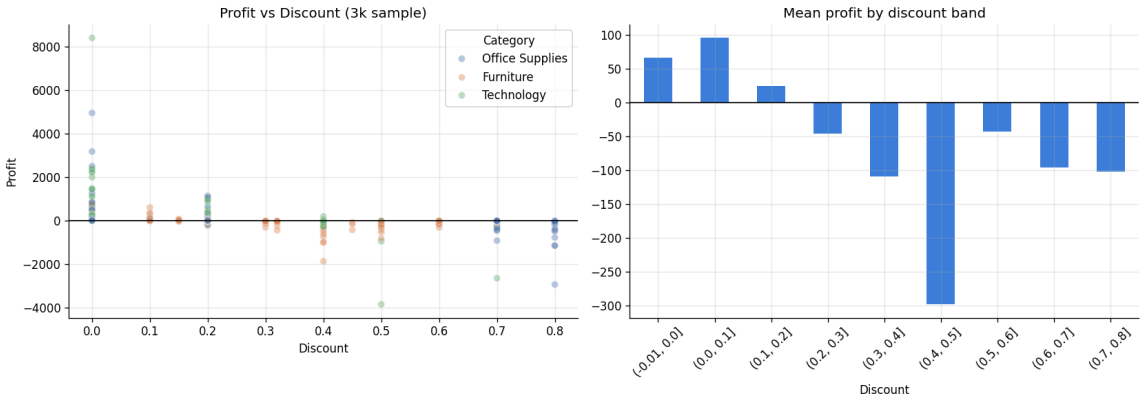

Figure 2.6: Left: line-level profit vs. discount (3,000-row sample), coloured by category — profit is scattered near zero at low discount and swings sharply negative as discount grows. Right: mean profit by discount band, crossing zero between the 10–20% and 20–30% bands and falling steeply beyond 30%.

Quantitatively, the loss rate ($\Pr(\text{Profit} < 0)$) is **exactly 0%** for undiscounted lines and **96.8%** for lines discounted 30% or more, against an overall loss rate of 18.7% across all 9,994 lines — discount depth is close to a deterministic switch, not a mild risk factor. Because the relationship is a threshold effect rather than linear, the raw Pearson correlation between `Discount` and `Profit` (visible in Fig. 2.7) understates its practical importance; this motivates using the categorical `discount_bucket` feature (rather than relying on a linear term alone) in Chapter 3 and **??**.

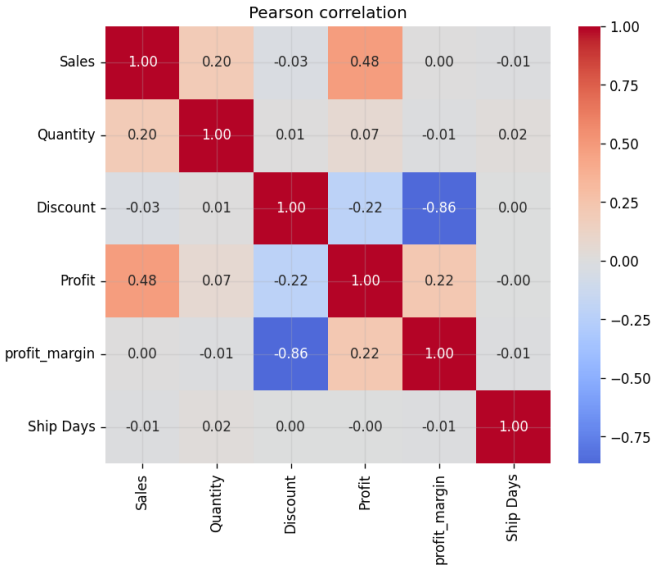

Figure 2.7: Pearson correlation (Eq. (2.2)) among the core numeric fields. `Sales`–`Profit`: $\rho \approx 0.48$; `Discount`–`Profit`: $\rho \approx -0.22$ (negative but, per Fig. 2.6, understating the non-linear effect); `Quantity` is nearly uncorrelated with profit.

2.3.5 Seasonality

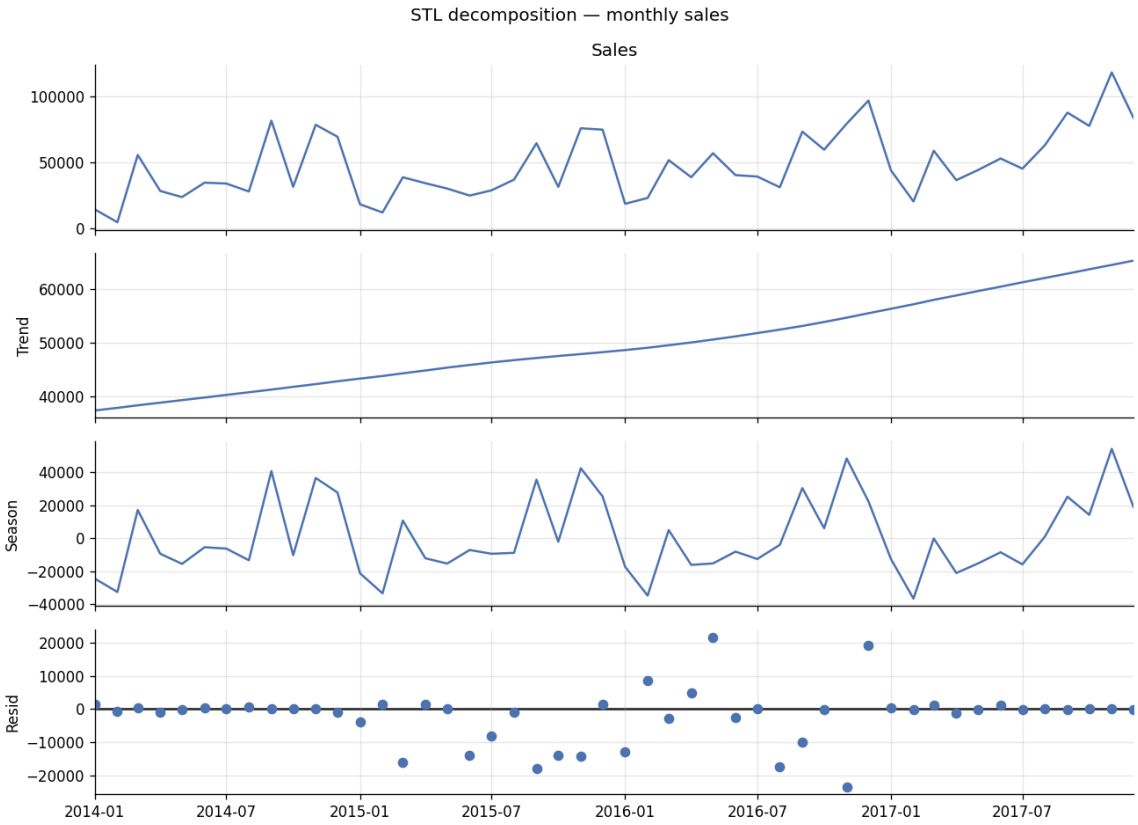

Figure 2.8: STL decomposition (Eq. (2.3), period 12) of monthly sales: a clear upward trend across the four years and a recurring within-year seasonal pattern that peaks late in the year, leaving a comparatively small, structure-free remainder.

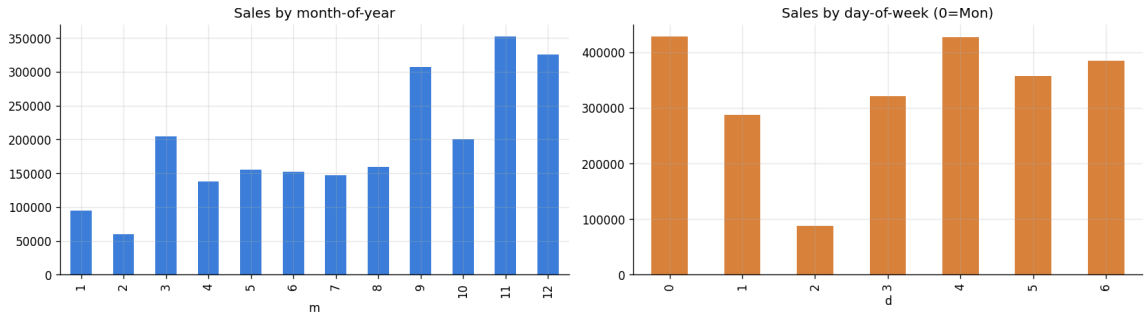

Figure 2.9: Sales aggregated by month-of-year (left) and day-of-week (right, 0 = Monday). November–December is by far the strongest period; day-of-week shows only mild variation.

2.3.6 Customers and products

The top-10 customers by sales (Table 2.2) generate between \$12k and \$25k each, but profitability is uneven within that list — *Sean Miller* is the single largest customer by revenue (\$25,043) yet is net loss-making (−\$1,981), while *Tamara Chand* generates comparable revenue at a much healthier margin. The same pattern repeats at product level: the *Cisco TelePresence System EX90* and two *GBC* binding systems are top-10 by revenue but net loss-making, again tracing back to discount depth on high-ticket Technology and Furniture items.

Table 2.2: Top 5 customers by total sales.

Customer	Sales ($)	Profit ($)	Orders
Sean Miller	25,043	−1,981	5
Tamara Chand	19,052	8,981	5
Raymond Buch	15,117	6,976	6
Tom Ashbrook	14,596	4,704	4
Adrian Barton	14,474	5,445	10

2.4 Interpretation

Three findings from this chapter shape every subsequent stage of the report. First, the data requires *no* missing-value or duplicate handling, so all modelling error downstream is genuinely about the signal, not data-cleaning artefacts. Second, **discount depth, not category or region, is the primary lever on profitability** — a threshold effect around 30% discount that is far stronger than any single-variable linear correlation suggests, which is why `discount_bucket` (Chapter 3) and the corresponding permutation-importance results (**??**) recur as the dominant signal throughout the rest of the report. Third, the pronounced trend and December seasonality in Fig. 2.8 is exactly the structure a seasonal model must capture, and directly motivates the choice of Holt-Winters and SARIMA — both explicitly seasonal — over simpler alternatives in Chapter 4.

Chapter 3

Feature Engineering

3.1 Theory

3.1.1 Data leakage

Definition 3.1.1 (Leakage)**.** *A feature $x^{(j)}$ used to predict target y_i for observation i* leaks *if it is constructed, even indirectly, from information that would not be available at the moment a real prediction for i has to be made — most commonly, information from i's own future or from i itself when y_i is a deterministic function of it.*

Leakage is the single most common cause of a model that performs excellently offline and fails in production, because the leaked signal is unavailable (or has a different distribution) at serving time. Two concrete leakage risks are present in this dataset and are handled explicitly:

(a) **Target leakage**: `profit_margin` = Profit/Sales is definitionally almost the target (`Profit`) rescaled, so it is computed only for exploratory plots (Chapter 2) and *never* included in the supervised feature matrix $\mathbf{X}$.

(b) **Temporal leakage in customer aggregates**: a naive "customer's total historical sales" feature computed with a normal `groupby(...).sum()` would, for order i, include sales from orders placed *after* i — information a real-time system could not have. We instead compute, for customer c and a chronologically ordered sequence of their orders $i_1 < i_2 < \dots$,

$$h_k = f\big(\{v_{i_1}, \dots, v_{i_{k-1}}\}\big), \qquad h_1 = f(\varnothing), \tag{3.1}$$

i.e. an *expanding window statistic shifted by one*, so h_k (the feature attached to the k-th order) is a function only of the customer's strictly *prior* orders. In `pandas` this is exactly `groupby(id)[col].apply(lambda s: s.shift().expanding(). agg(f))`, and it is verified empirically in Section 3.3 by checking that every customer's first order has a history of identically zero.

3.1.2 Cyclical (circular) encoding of periodic variables

Calendar fields such as month-of-year $m \in \{1, \dots, 12\}$ or day-of-week $d \in \{0, \dots, 6\}$ are *circular*: December (12) is adjacent to January (1), not maximally distant from it, yet a plain integer encoding implies exactly that. Passing m into a linear or distance-based model as a raw integer therefore injects a false ordinal distance. The standard fix is to embed the period on the unit circle,

$$m_{\sin} = \sin\!\left(\frac{2\pi m}{12}\right), \qquad m_{\cos} = \cos\!\left(\frac{2\pi m}{12}\right), \tag{3.2}$$

so that the Euclidean distance between two months' encodings, $\sqrt{(m_{\sin} - m'_{\sin})^2 + (m_{\cos} - m'_{\cos})^2} = 2\left|\sin\frac{\pi(m-m')}{12}\right|$, is a monotone function of their true circular distance and wraps correctly around the year boundary. Tree ensembles (used for the best-performing models in **??** and Chapter 4) do not strictly require this — they can split m arbitrarily and effectively rediscover adjacency — but linear and distance-based components still benefit, and the encoding is cheap and included uniformly.

3.1.3 Categorical encoding

For a categorical field with levels $\{c_1, \ldots, c_K\}$, *one-hot encoding* maps a value c_k to the k-th standard basis vector $\mathbf{e}_k \in \{0,1\}^K$, avoiding the false ordinal structure that a plain integer code would impose on an unordered category, at the cost of adding $K-1$ columns (one level dropped or absorbed as the reference via collinearity handling). With K large or with many rare levels — the Superstore data has 49 U.S. states, most with a handful of orders — this inflates dimensionality and lets a model overfit to noise in low-count categories. We mitigate this by collapsing any level occurring fewer than 20 times into a single `"infrequent"` bucket before one-hot expansion (scikit-learn's `OneHotEncoder(min_frequency= 20)`), trading a small amount of resolution for materially lower variance in the corresponding coefficient/split estimates.

3.1.4 RFM analysis

RFM (Recency, Frequency, Monetary) is a classical customer segmentation heuristic from direct-marketing analytics. For customer c and an analysis snapshot date τ,

$$R_c = \tau - \max\{\text{Order Date} : \text{orders of } c\}, \tag{3.3}$$

$$F_c = \left|\{\text{distinct orders of } c\}\right|, \tag{3.4}$$

$$M_c = \sum_{\text{orders of } c} \text{Sales}. \tag{3.5}$$

Each of R, F, M is independently discretised into quintiles (1 = worst, 5 = best; R is inverted since *smaller* recency is better), and an overall score $\text{RFM}_c = r_c + f_c + m_c \in [3, 15]$ ranks customers by a simple, interpretable combination of how recently, how often, and how much they buy — used here purely as a descriptive segmentation (Section 3.3), not as a model feature (to avoid mixing a snapshot-time construct into the leakage-safe, per-order feature matrix).

3.1.5 Right-skew and the log transform, revisited

As introduced in Chapter 2, $\log(1+x)$ is applied to `Sales` as an explicit feature (`log_sales`) rather than only a plotting device, because linear/coefficient-based models benefit from the reduced skew and more homoscedastic residual structure it induces, while the tree-based models used in **??** are monotone-transform invariant and are unaffected by whether it is present.

3.2 Method

All logic lives in `src/features.py` and is exercised end to end in `03_feature_engineering.ipynb`. `add_targets` derives `profit_margin` and `is_loss` (kept out of $\mathbf{X}$ as above); `add_date_features` derives calendar parts and the sin/cos pairs of Eq. (3.2); `add_line_features` derives `price_per_unit`, `log_sales`, `is_discounted`, and a 5-level `discount_bucket` (*none / low / mid / high / extreme*, edges at 0, 15%, 30%, 50%); `add_customer_history` implements the expanding-shift computation of Eq. (3.1) for prior order count, prior sales sum/mean, prior loss rate, and days since the

customer's first order. `make_supervised` assembles the final $n \times p$ matrix $\mathbf{X}$ (26 numeric + 6 categorical columns, categoricals left as raw strings for the modelling pipeline's own `OneHotEncoder` in **??**) and the two targets $\mathbf{y}_{\text{reg}} = \text{Profit}$, $\mathbf{y}_{\text{clf}} = \text{is\_loss}$.

3.3 Results

3.3.1 Feature matrix and leakage check

Building the full feature frame adds 26 engineered columns to the 21 raw ones, and the final supervised matrix is $\mathbf{X} \in \mathbb{R}^{9994 \times 32}$ with **zero missing values**. The leakage sanity check passes exactly as designed: across all 793 customers' first-ever order line, `cust_prior_lines` and `cust_prior_sales_sum` are identically 0 — no customer's first transaction sees any of their own future purchase history.

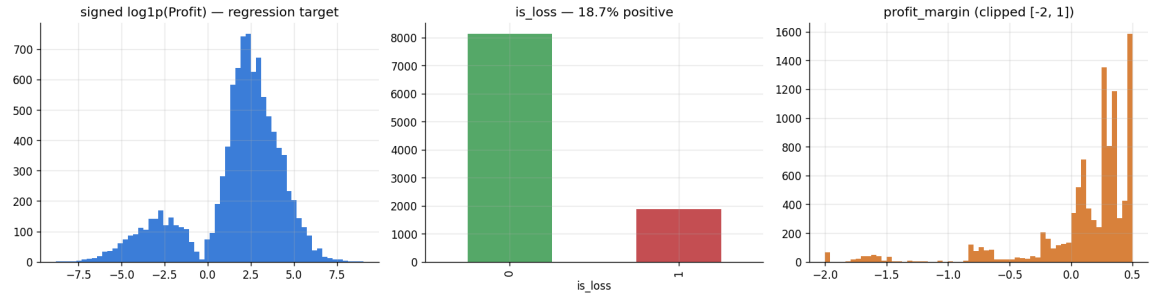

Figure 3.1: The two supervised targets and the exploratory `profit_margin`: signed log-profit (left), the binary `is_loss` class balance — 18.7% positive (middle), and margin distribution (right).

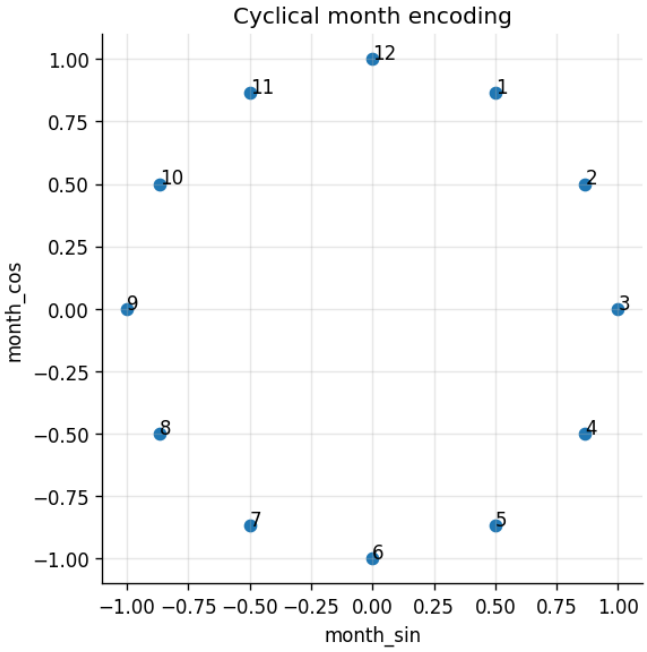

Figure 3.2: The 12 months placed on the unit circle by Eq. (3.2): December and January are adjacent, as they should be, unlike under a raw integer encoding.

3.3.2 Discount buckets

Table 3.1 operationalises the threshold effect found in Chapter 2: mean profit and loss rate degrade monotonically across buckets, flipping sign between *mid* and *high*, and the *extreme* bucket (discount $\geq 50\%$) is loss-making on **every single line** in the data.

Table 3.1: Line-level statistics by `discount_bucket`.

Bucket	n	Mean profit ($)	Loss rate	Mean margin
none (0%)	4,798	66.90	0.0%	34.0%
low (0–15%)	146	71.56	14.4%	11.2%
mid (15–30%)	3,884	20.59	18.3%	16.0%
high (30–50%)	310	−156.28	91.6%	−29.6%
extreme (≥50%)	856	−89.44	**100.0%**	−113.9%

3.3.3 Customer history features

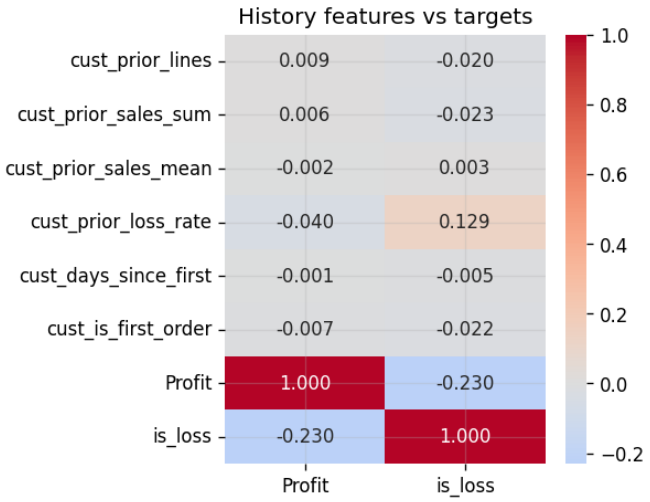

Figure 3.3: Correlation of the leakage-safe customer-history features with the two targets. `cust_prior_loss_rate` is the strongest of the six ($\rho = 0.129$ with `is_loss`) — a customer who has previously bought at deep discounts is more likely to do so again.

The other history features (prior line count, prior sales sum/mean, days since first order) correlate only weakly ($|\rho| < 0.03$) with either target at the line level — unsurprising, since a given customer's profit outcome on *this* line is driven mostly by *this* line's own discount, not by how much they have historically spent.

3.3.4 RFM segmentation

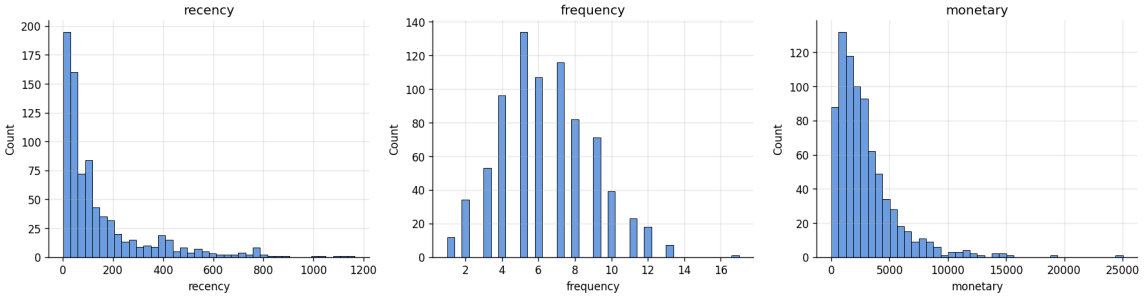

Figure 3.4: Distributions of Recency, Frequency and Monetary value across the 793 customers.

Table 3.2: RFM segments (score bands on $R + F + M \in [3, 15]$).

Segment	Customers	Avg. Monetary ($)	Avg. Frequency	Avg. Recency (days)
At risk	196	989	3.7	329
Needs attention	219	2,160	5.6	138
Loyal	254	3,869	7.4	75
Champions	124	5,221	9.4	28

The four segments form a coherent gradient: Champions buy 2.5× more often and spend 5.3× more per customer on average than the At-risk segment, and were last seen 12× more recently — exactly the monotone structure RFM is designed to expose.

3.4 Interpretation

The leakage-safe construction of the customer-history features is the methodologically important result of this chapter: it guarantees that the 2017 hold-out evaluation in **??** is a fair test of generalisation to a genuinely unseen future, not an artefact of features that implicitly encode the answer. Substantively, the discount-bucket table makes the EDA finding operational and almost mechanical: *extreme* discounting is loss-making with effective certainty in this dataset, which is precisely the signal the classifier of **??** is able to exploit to reach a PR-AUC of 0.96. RFM segmentation, while not fed into the predictive models, gives an independent, interpretable customer lens that a retail analytics team could pair with the profit/loss model output — e.g. prioritising discount governance on At-risk/Needs-attention customers, where a marginal loss is more likely to also coincide with a customer who is not being retained by it anyway.

Chapter 4

Forecasting Monthly Sales

This is the most theory-heavy chapter of the report: forecasting a short (48-point) monthly series correctly requires care about stationarity, model identification, and — critically — an evaluation protocol that respects time order, none of which arise in the i.i.d. supervised setting of **??**.

4.1 Theory

4.1.1 Stationarity

Definition 4.1.1 (Weak stationarity)**.** *A process* $\{y_t\}$ *is* weakly (covariance) stationary *if* $\mathbb{E}[y_t] = \mu$ *is constant in* t*,* $\mathrm{Var}(y_t) = \sigma^2$ *is constant in* t*, and* $\mathrm{Cov}(y_t, y_{t+h})$ *depends on the lag* h *but not on* t *itself.*

Classical linear time-series models (AR, MA, ARMA) are defined for stationary processes; a series with a trend or a growing seasonal amplitude violates the definition and must be transformed — typically by *differencing* — before such a model applies. The backshift (lag) operator B, $By_t = y_{t-1}$, lets a d-th order difference be written compactly as $\Delta^d y_t = (1-B)^d y_t$; a *seasonal* difference of period s is $\Delta_s y_t = (1 - B^s)y_t = y_t - y_{t-s}$.

4.1.2 The augmented Dickey–Fuller test

The ADF test formalises "does this series need differencing" as a hypothesis test on the regression

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^{k} \delta_i \Delta y_{t-i} + \varepsilon_t, \tag{4.1}$$

with null hypothesis $H_0 : \gamma = 0$ (a unit root — the series is a random walk, non-stationary) against $H_1 : \gamma < 0$ (stationary). The test statistic is the ordinary t-statistic for $\hat{\gamma}$, but its null distribution is *not* the usual Student-t (it is skewed left by the non-standard behaviour of a random walk's sample variance), so Dickey and Fuller's own tabulated critical values (or the software's p-value approximation) must be used rather than a normal-theory table.

4.1.3 Autocorrelation and the Box–Jenkins identification

The autocorrelation function $\rho(h) = \mathrm{Corr}(y_t, y_{t+h})$ and partial autocorrelation function ϕ_{hh} (the correlation between y_t and y_{t+h} after regressing out $y_{t+1}, \ldots, y_{t+h-1}$) are the classical diagnostic

pair for identifying ARMA order: an AR(p) process has a PACF that cuts off after lag p and an ACF that decays geometrically, while an MA(q) process has the mirror-image behaviour. In practice for a short, strongly seasonal series like ours, ACF/PACF are used qualitatively (Fig. 4.2) to corroborate, not to mechanically dictate, the model order.

4.1.4 Exponential smoothing (Holt–Winters)

Simple exponential smoothing forecasts the next value as a geometrically-weighted average of the past, $\hat{y}_{t+1} = \alpha y_t + (1-\alpha)\hat{y}_t$. **Holt–Winters** extends this with a trend state b_t and a seasonal state s_t (period $L = 12$). The additive form used here is

$$\ell_t = \alpha(y_t - s_{t-L}) + (1-\alpha)(\ell_{t-1} + b_{t-1}), \tag{4.2}$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}, \tag{4.3}$$

$$s_t = \gamma(y_t - \ell_t) + (1-\gamma)s_{t-L}, \tag{4.4}$$

$$\hat{y}_{t+h} = \ell_t + h\,b_t + s_{t-L+((h-1) \bmod L)+1}, \tag{4.5}$$

with smoothing parameters $\alpha, \beta, \gamma \in [0,1]$ chosen (by the fitting routine) to minimise the in-sample sum of squared one-step forecast errors. Level ℓ_t tracks the de-seasonalised local mean, b_t the local trend, and s_t a smoothly-updating length-L seasonal profile; the h-step forecast Eq. (4.5) linearly extrapolates the trend and re-applies the appropriate seasonal index. This additive form is appropriate when the seasonal swing has roughly constant absolute size rather than growing proportionally with the level (a multiplicative alternative replaces the $+s$ terms with $\times s$ terms), consistent with Fig. 2.8 where the seasonal amplitude does not visibly balloon with the trend.

4.1.5 SARIMA

A seasonal ARIMA model, SARIMA$(p,d,q)(P,D,Q)_s$, generalises ARMA by combining non-seasonal and seasonal autoregressive/moving-average polynomials applied to a series differenced both regularly (d times) and seasonally (D times, period s):

$$\phi_p(B)\,\Phi_P(B^s)\,(1-B)^d(1-B^s)^D y_t = \theta_q(B)\,\Theta_Q(B^s)\,\varepsilon_t, \tag{4.6}$$

where $\phi_p(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$ (and analogously Φ_P, θ_q, Θ_Q for the seasonal AR, non-seasonal MA and seasonal MA polynomials respectively) and ε_t is white noise. We fit SARIMA$(1,1,1)(1,1,0,12)$: one non-seasonal AR and MA term, one seasonal AR term, one regular and one seasonal (period-12) difference, and no seasonal MA term — a compact specification chosen to be identifiable on 48 points while still capturing both the trend ($d = 1$) and the year-over-year seasonal shape ($D = 1$, period 12) directly, rather than relying on Holt–Winters' exponential-smoothing-style seasonal update. Coefficients are estimated by (quasi-)maximum likelihood via the Kalman filter (as implemented in `statsmodels`'s state-space `SARIMAX`), which also yields exact prediction variances and hence analytic confidence intervals for the forecast.

4.1.6 Forecast accuracy metrics

For actuals $y_1, \ldots, y_h$ and forecasts $\hat{y}_1, \ldots, \hat{y}_h$ over a horizon of length h:

$$\mathrm{MAE} = \frac{1}{h}\sum_{i=1}^{h} |y_i - \hat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{h}\sum_{i=1}^{h}(y_i - \hat{y}_i)^2}, \tag{4.7}$$

$$\mathrm{MAPE} = \frac{100\%}{h}\sum_{i=1}^{h}\left|\frac{y_i - \hat{y}_i}{y_i}\right|. \tag{4.8}$$

MAE is in the original (dollar) units and treats all errors linearly; RMSE is also in dollars but penalises large errors quadratically, so it is more sensitive to occasional big misses (e.g. an under-forecast December); MAPE is scale-free and hence comparable across series of different magnitude (used here to compare the whole-store series against per-category series of very different size), at the cost of being undefined/unstable when y_i is near zero and of asymmetrically penalising over- vs. under-forecasts.

4.1.7 Rolling-origin (walk-forward) backtesting

A single train/test split, as used for the i.i.d. models of **??**, is a poor evaluation protocol for a 48-point time series: it tests the model at exactly one forecast origin, so its score has high variance and cannot be trusted to represent typical future performance. **Rolling-origin** (a.k.a. walk-forward, or expanding- window) backtesting instead repeats the fit/forecast cycle at multiple origins $o_1 < o_2 < \cdots < o_K$: at each o_k, the model is trained only on $y_1, \ldots, y_{o_k}$ and scored against the next h true values $y_{o_k+1}, \ldots, y_{o_k+h}$, and the reported metric is the average across the K folds. This never trains on the future relative to any fold's own test window and is the time-series analogue of K-fold cross-validation, with the essential difference that folds must respect temporal order rather than being drawn at random.

4.1.8 Residual whiteness: the Ljung–Box test

A well-specified model should leave residuals $\hat{\varepsilon}_t$ that are approximately white noise (no remaining autocorrelation to exploit). The Ljung–Box statistic

$$Q_{LB} = n(n+2)\sum_{k=1}^{K}\frac{\hat{\rho}_k^2}{n-k} \tag{4.9}$$

(with $\hat{\rho}_k$ the residual sample autocorrelation at lag k) is asymptotically χ^2_K under the null of no autocorrelation up to lag K; a large p-value (failure to reject) is the desired outcome, indicating the model has not left exploitable structure behind.

4.2 Method

The monthly series $y_t = \sum_{\text{lines in month } t} \text{Sales}$ (`src.features.monthly_sales`) has $T = 48$ points, 2014-01 through 2017-12. We hold out the last 12 months as a single test window, fit four forecasters on the code interface `fit(train)` / `predict(h)` (`src/forecasting.py`) — a seasonal-naive baseline, additive Holt–Winters, SARIMA$(1,1,1)(1,1,0,12)$, and a recursive random-forest regressor on lag/rolling-mean/seasonal features (`LagRegressor`) — and separately run an 8-fold rolling-origin backtest (`min_train`=24, horizon=3, step=3). The winning point forecaster is then refit on the full 48 months and projected 12 months ahead; SARIMA additionally supplies an 80% analytic confidence band via its Kalman-filter forecast variance.

4.3 Results

4.3.1 Series and stationarity

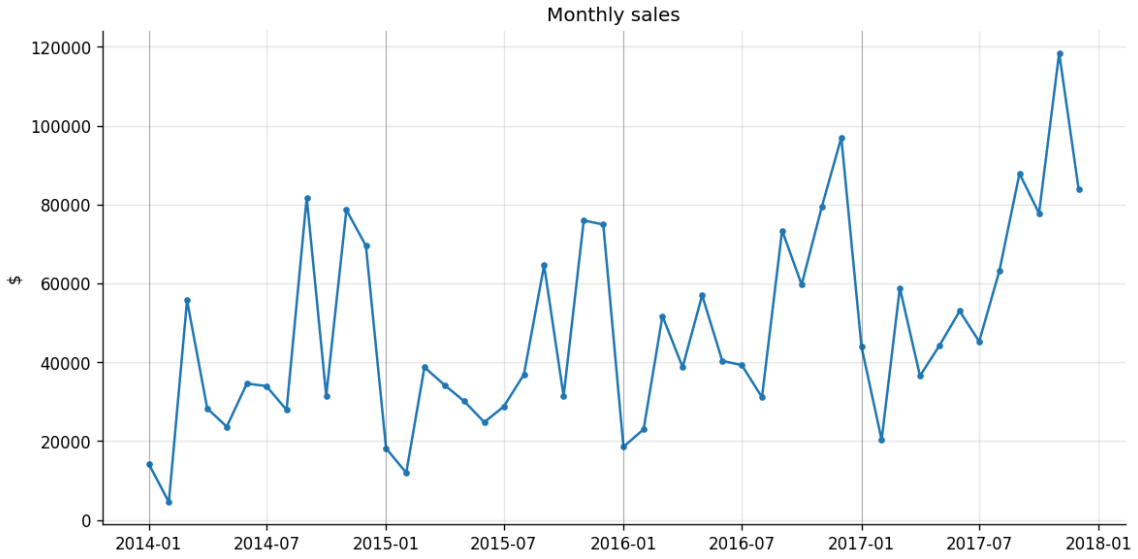

Figure 4.1: Monthly sales, 2014–2017. Clear multi-year growth and a recurring within-year cycle.

Table 4.1 reports the ADF test (Eq. (4.1)) on the level series, its first difference, and its seasonal (lag-12) difference.

Table 4.1: ADF unit-root test, H_0: series has a unit root.

Series	Test statistic	p-value
Level y_t	-4.494	0.0002
First difference Δy_t	-9.058	<0.0001
Seasonal difference $\Delta_{12} y_t$	-1.482	0.5425

The level series already rejects a pure unit root, and first-differencing rejects it far more decisively — but the *seasonal-only* difference does **not** (p = 0.54): removing only the year-over-year level shift is not enough to leave a stationary residual, because the within-year *shape* of the cycle (not just its level) still carries structure. This is exactly the case for using a model with an explicit, separately-estimated seasonal component — Holt–Winters' s_t state, or SARIMA's combined regular-*and*-seasonal differencing (Eq. (4.6), d=1, D=1) — rather than relying on a single seasonal difference alone.

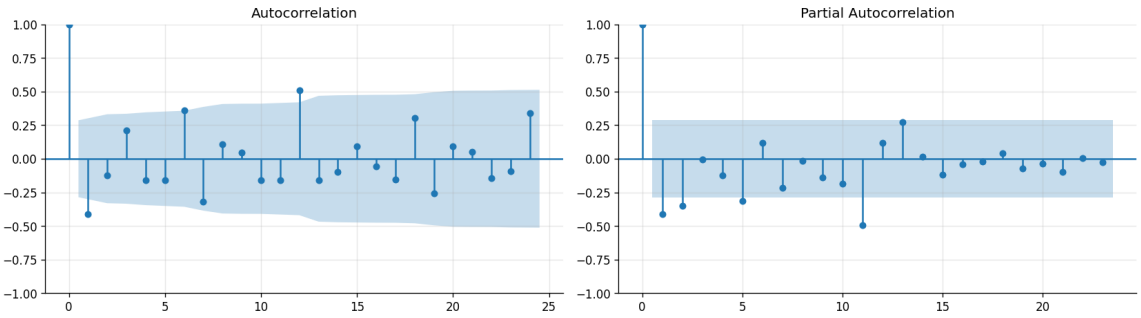

Figure 4.2: ACF (left) and PACF (right) of Δy_t. A significant spike near lag 12 in both is consistent with the seasonal structure seen directly in Figs. 2.8 and 4.1.

4.3.2 Hold-out comparison

Table 4.2: 12-month hold-out accuracy (train on months 1–36, test on 37–48).

Model	MAE ($)	RMSE ($)	MAPE
Holt–Winters	**11,456**	12,544	**22.6%**
SARIMA(1,1,1)(1,1,0,12)	13,411	15,989	26.7%
Lag-regressor (RF)	13,484	16,791	24.1%
Seasonal-naive baseline	15,468	18,986	24.5%

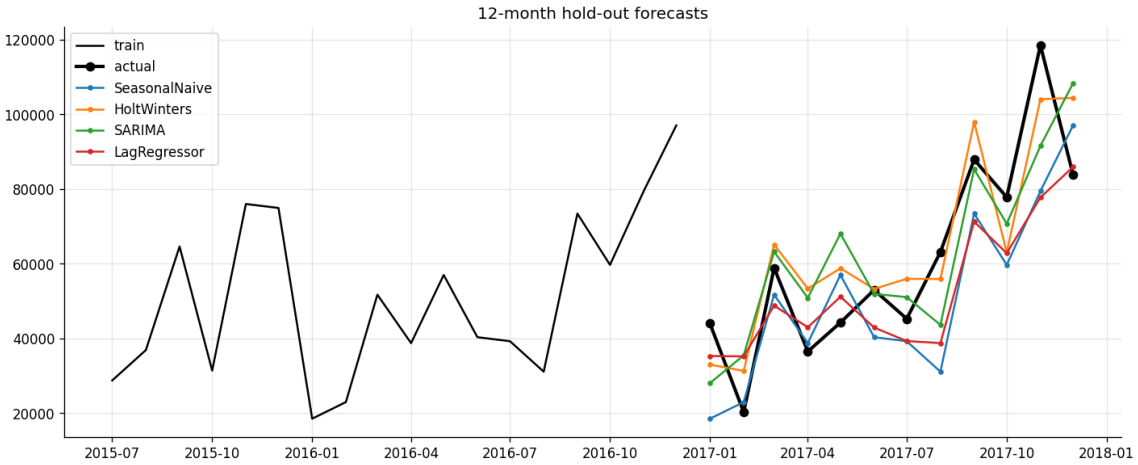

Figure 4.3: 12-month hold-out: all four models' forecasts against the realised sales path.

All three fitted models beat the seasonal-naive baseline on MAE and RMSE; Holt–Winters is the clear winner on this single split.

4.3.3 Rolling-origin backtest

Table 4.3: 8-fold rolling-origin backtest (`min_train`=24, horizon=3, step=3), metrics averaged across folds.

Model	MAE ($)	RMSE ($)	MAPE	Folds
SARIMA	**11,612**	13,373	23.9%	8
Holt–Winters	12,314	14,104	25.1%	8
Lag-regressor (RF)	13,660	15,527	27.2%	8
Seasonal-naive baseline	13,994	16,207	24.9%	8

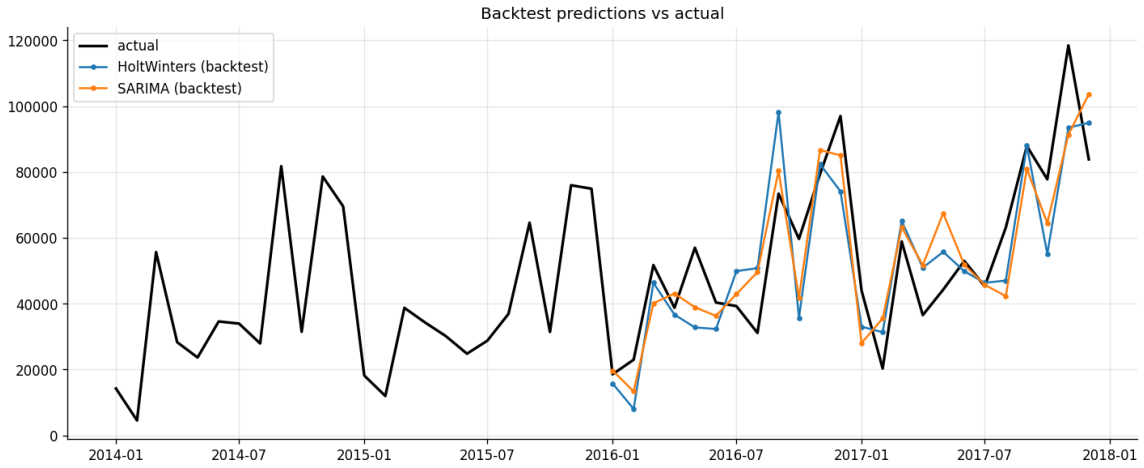

Figure 4.4: Rolling-origin backtest predictions (Holt–Winters and SARIMA) vs. actual, across all 8 folds.

The ranking flips: with more forecast origins averaged, SARIMA edges out Holt–Winters ($11,612 vs. $12,314 MAE), though the two remain within about 6% of each other and both clearly beat the naive baseline and the lag-regressor. This is a useful illustration of exactly the risk the theory section warns about: the single hold-out split (Table 4.2) and the 8-fold backtest (Table 4.3) disagree on which of the top two models is "best", and the backtest, having 8 origins rather than 1, is the more trustworthy estimate of typical future accuracy.

4.3.4 Final forecast

Given the closeness of the two models and their complementary strengths (Holt–Winters: best single-split point forecast; SARIMA: best backtest average *and* a principled analytic interval), the final projection uses **Holt–Winters, refit on all 48 months, for the point forecast** and **SARIMA for an 80% confidence band**.

Figure 4.5: History plus the 12-month-ahead forecast: Holt–Winters point forecast and the SARIMA 80% interval.

The next 12 months are projected at **$914,903**, against $733,215 realised in the last full year (2017) — a projected **+24.8%** year-on-year. Table 4.4 breaks this down by product category (each fit with its own Holt–Winters model on that category's monthly series).

Table 4.4: Per-category 2018 projection (Holt–Winters).

Category	2017 actual ($)	2018 projected ($)	Growth
Office Supplies	246,097	326,734	**+32.8%**
Furniture	215,387	245,463	+14.0%
Technology	271,731	272,220	+0.2%

4.3.5 Residual diagnostics

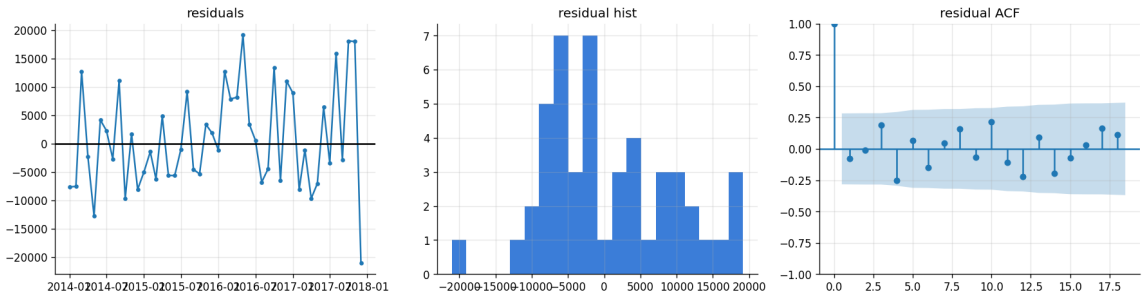

Figure 4.6: Holt–Winters in-sample residuals: time plot, histogram, and ACF.

The Ljung–Box statistic (Eq. (4.9)) at lag 12 is $Q_{LB} = 16.11$, $p = 0.186$ — we fail to reject the null of no residual autocorrelation, i.e. the fitted Holt–Winters model has not left obviously exploitable structure in its residuals, a reasonable (if not definitive, given $T = 48$) check of adequacy.

4.4 Interpretation

Three points carry forward. First, the ADF results (Table 4.1) justify the modelling choice made throughout this chapter: the series' seasonal *shape*, not only its trend, needs explicit esti-

mation, which is exactly what both Holt–Winters' seasonal state s_t and SARIMA's $(P, D, Q)_{12}$ terms provide, and is why a naive linear-trend-only model is not attempted. Second, the disagreement between the single hold-out split and the 8-fold rolling backtest on which of Holt–Winters/SARIMA is "better" is not noise to be explained away — it is the expected behaviour of a short series with few independent forecast origins, and is the reason this report reports *both* evaluations rather than picking whichever looks best. Third, the practical takeaway for the business is directional, not a claim of precision: total sales are projected up by roughly a quarter next year, but growth is sharply uneven by category — Office Supplies is accelerating, Technology is flat — which is a materially different and more actionable statement than a single store-wide growth number, and motivates category-aware inventory/staffing planning rather than a uniform +25% assumption.